

POST-QUANTUM FIRMWARE AUTHENTICATION: DESIGN AND IMPLEMENTATION OF A QUANTUM-RESISTANT SECURE BOOT MECHANISM

BITS ZG628T: Dissertation

by

**Kiruthik Prakash J
2024HT01586**

Dissertation work carried out at
Qualcomm India Private Limited, Hyderabad

Submitted in partial fulfilment of M.Tech in Embedded Systems
degree programme

Under the Supervision of

**Deepak Kumar
Senior Lead Software Engineer
Qualcomm India Private Limited, Hyderabad**



**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE
PILANI (RAJASTHAN)
SEPTEMBER 2026**

CERTIFICATE

This is to certify that the dissertation entitled “**Post-Quantum Firmware Authentication: Design and Implementation of a Quantum-Resistant Secure Boot Mechanism**” and submitted by **Kiruthik Prakash J (ID: 2024HT01586)** in partial fulfilment of the requirement of **M.Tech in Embedded Systems** of BITS Pilani, embodies the work done by him under my supervision at **Qualcomm India Private Limited, Hyderabad**.

/s/ Deepak Kumar

Deepak Kumar
Supervisor
Senior Lead Software Engineer
Qualcomm India Private Limited

Date: 19/09/2026
Place: Hyderabad

/s/ Kiruthik Prakash J

Kiruthik Prakash J
ID: 2024HT01586
M.Tech Embedded Systems
BITS Pilani

Date: 19/09/2026
Place: Hyderabad

TABLE OF CONTENTS

1. MODULES IN POST-QUANTUM SECURE BOOT SYSTEM	3
1.1 Core Cryptographic Verification Engine	4
1.2 IoT & Microcontroller Target: MCUboot Bootloader Module	5
1.3 Embedded Linux Target: Das U-Boot FIT Verification Module	5
1.4 Enterprise / Server Target: EDKII UEFI SecurityPkg Module	6
1.5 Hardware Root-of-Trust (RoT) & eFuse Key Binding Module	6
1.6 Offline Firmware Signing & Container Tooling Module	6
1.7 Multi-Target Emulation & Automated Verification Harness	7
2. FUNCTIONAL BLOCK DIAGRAM & ARCHITECTURAL DESCRIPTION	8
2.1 System Architecture Overview	8
2.2 Secure Boot Execution Sequence & Cryptographic Flow	9
3. MAJOR TECHNICAL SPECIFICATIONS OF PQC SECURE BOOT	10
4.1 Zero Dynamic Memory Allocation (Zero-Malloc Policy)	11
4.2 Strict Static SRAM Budgeting (< 4 KB ROM Bounds)	11
4.3 Hardware Root-of-Trust Key Hash Binding & eFuse Storage	11
4.4 Tamper Robustness & Non-Negotiable Fault Rejection	11
4.5 Constant-Time Cryptographic Execution & Side-Channel Mitigation	12
5. EMPIRICAL BENCHMARKING, PROFILING & FUTURE PLAN	13
5.1 MCUboot on ARM Cortex-M4 Trade-Off Benchmark & Stack Profiling	13
5.2 Mid-Semester Progress Status (Plan of Work)	15
5.3 Remaining Tasks & Deliverables for Final Dissertation	16
6. ABBREVIATIONS	17
7. REFERENCES & LITERATURE REVIEW	19

List of Figures

Figure 1: Modular Architecture of Post-Quantum Secure Boot System	8
Figure 2: Functional Block Diagram & Secure Boot Execution Flow	9

List of Tables

Table 1: Technical Specifications & Cryptographic Parameters	10
Table 2: MCUboot ARM Cortex-M4 Trade-Off Benchmark (Classical vs PQC)	13
Table 3: Dissertation Plan of Work & Mid-Semester Status	15
Table 4: Table of Abbreviations & Acronyms	17

1. MODULES IN POST-QUANTUM SECURE BOOT SYSTEM

The Post-Quantum Cryptography (PQC) Secure Boot infrastructure (pqc-boot) is architected as a modular, hardware-agnostic, and zero-dynamic-memory framework designed to authenticate bare-metal firmware across the complete embedded spectrum—from resource-constrained microcontrollers to enterprise multi-core servers. The complete system consists of seven primary functional modules:

- (a) Core Cryptographic Verification Engine (pqc_crypto, ml_dsa, sphincs_plus, lms, sha256, shake256)
- (b) IoT & Microcontroller Bootloader Module (MCUboot with PQC TLV tags on ARM Cortex-M4)
- (c) Embedded Linux Bootloader Module (Das U-Boot with FIT Device Tree Blob on RISC-V 64-bit)
- (d) Enterprise / Server Firmware Module (EDKII / UEFI SecurityPkg DxeImageVerificationLib on Cortex-A57 SMP)
- (e) Hardware Root-of-Trust (RoT) & eFuse Key Binding Module (rot_key)
- (f) Offline Firmware Signing & Container Tooling Module (imgtool, mkimage, sign_firmware.py)
- (g) Multi-Target Emulation & Automated Verification Harness (tests, QEMU platform runners)

1.1 Core Cryptographic Verification Engine

The core cryptographic engine (located in the firmware module `firmware/src/pqc_crypto.c`) serves as the centralized, static-memory cryptographic service layer. It encapsulates NIST-standardized and RFC-specified post-quantum signature schemes alongside required symmetric primitives:

- **ML-DSA-44 (NIST FIPS 204):** Module-Lattice-Based Digital Signature Algorithm (formerly Dilithium2). Operates over polynomial rings $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ with modulus $q = 8,380,417$. The engine defines public key structures (1,312 bytes), private key structures (2,560 bytes), and signature structures (2,420 bytes). Forward and inverse Number Theoretic Transforms (NTT), Montgomery modular reductions, and uniform polynomial sampling routines are implemented.
- **SPHINCS+ / SLH-DSA (NIST FIPS 205):** Stateless Hash-Based Digital Signature Algorithm (SLH-DSA-128f parameter set). Relies solely on the collision resistance of cryptographic hash functions without algebraic lattice assumptions. Utilizes Winternitz One-Time Signatures (WOTS+) and Forest of Random Subsets (FORS) multi-layer hypertree constructions with a compact

32-byte public key (PK.seed || PK.root) and an allocated signature buffer of up to 18,000 bytes.

- **LMS / LMOTS (RFC 8554 / RFC 8708):** Leighton-Micali Stateful Hash-Based Signature scheme using LMOTS-SHA256_N32_W4 one-time signatures and LMS_SHA256_M32_H10 Merkle trees. Features a 56-byte public key containing the tree type, LMOTS type, 16-byte identifier I, and 32-byte root node K.
- **Symmetric Hashing Primitives:** Implements zero-heap SHA-256 (FIPS 180-4) and SHAKE-256 Keccak extensible-output functions (FIPS 202). In the current milestone, signature authentication combines pre-hashing ($\mu = \text{SHA256}(\text{PK} \parallel \text{msg})$), challenge seed derivation ($c_{\text{tilde}} = \text{SHAKE256}(\mu, 32)$), and active polynomial NTT arithmetic.

1.2 IoT & Microcontroller Target: MCUboot Bootloader Module

The MCUboot module (real_world/mcuboot/) integrates post-quantum firmware validation into the leading open-source 32-bit microcontroller secure bootloader:

- **PQC Type-Length-Value (TLV) Header Extensions:** Defined custom TLV record identifiers in boot/bootutil/include/bootutil/image.h: IMAGE_TLV_ML_DSA_44 (0x80), IMAGE_TLV_SPHINCS_PLUS (0x81), IMAGE_TLV_LMS (0x82), and IMAGE_TLV_PQC_PUBKEY (0x88).
- **Zero-Allocation Verification Hook:** Implemented bootutil_pqc.c, hooking bootutil_pqc_verify_ml_dsa_44() directly into bootutil_img_validate(). The routine traverses image TLVs, extracts public key digests, verifies RoT bindings, and authenticates image payloads without heap usage.
- **Host Signing Utility (imgtool):** Extended scripts/imgtool/main.py and keys/pqc.py to support the command imgtool keygen -k keys/ml_dsa_key.json -t ml-dsa-44 and automated trailer encapsulation during imgtool sign.

1.3 Embedded Linux Target: Das U-Boot FIT Verification Module

The Das U-Boot integration module (real_world/uboot/) incorporates PQC verification into the standard Flattened Image Tree (FIT) mechanism used across ARM and RISC-V embedded Linux deployments:

- **Crypto Dispatcher Registration:** Registered post-quantum signature handlers in boot/image-sig.c using the U-Boot driver macro U_BOOT_CRYPTO_ALGO(ml_dsa_44), U_BOOT_CRYPTO_ALGO(sphincs_plus), and U_BOOT_CRYPTO_ALGO(lms).
- **FIT Signature Verification Engine:** Created lib/pqc/pqc_fit_verify.c and lib/pqc/pqc-verify.c. The verification driver extracts image nodes from device tree blobs (.itb), validates the property algo = "sha256,ml-dsa-44", fetches public keys referenced by key-name-hint, and executes verification.

- **Host mkimage Tool Integration:** Updated tools/Makefile and image-sig-host.c to compile PQC verification and signing engines directly into the host mkimage utility binary, enabling automated FIT image compilation via .its scripts.

1.4 Enterprise / Server Target: EDKII UEFI SecurityPkg Module

The EDKII / UEFI module (real_world/edk2/) addresses enterprise server architectures running 64-bit multi-core processors. Firmware authentication occurs during the Driver Execution Environment (DXE) phase of UEFI Secure Boot:

- **Post-Quantum Object Identifiers (OIDs):** Defined ASN.1 Object Identifiers in SecurityPkg/Include/Library/PqcVerify.h: `OID_ML_DSA_44` (2.16.840.1.101.3.4.3.17), `OID_SPHINCS_PLUS` (2.16.840.1.101.3.4.3.20), and `OID_LMS_HASH` (1.2.840.113549.1.9.16.3.17).
- **PKCS#7 Verification Integration:** Implemented `Pkcs7VerifyPqc.c` and integrated it into `DxeImageVerificationLib`. The module parses PE/COFF certificate tables, decodes Authenticode digital signatures, and authenticates .efi OS loader binaries against Root-of-Trust keys stored in UEFI Authenticated Variables (db).
- **Multi-Core SMP Reentrancy:** Eliminated static mutable globals from the DXE verification path, guaranteeing thread safety and reentrancy across multi-core symmetric multiprocessing (SMP) server nodes.

1.5 Hardware Root-of-Trust (RoT) & eFuse Key Binding Module

Because post-quantum public keys are significantly larger than classical keys (e.g., 1,312 bytes for ML-DSA-44 vs. 32 bytes for ECDSA P-256), physical on-chip One-Time Programmable (OTP) eFuse arrays cannot store raw PQC public keys directly. The RoT module (firmware/src/rot_key.c) addresses this physical constraint through a two-stage binding model:

1. **Hardware eFuse Hash Commitment:** A 256-bit SHA-256 hash of the authorized Root Public Key is burned into OTP eFuse storage or immutable Boot ROM constants.
2. **Header Public Key Verification:** During boot, the bootloader reads the full public key embedded in the firmware header, computes its SHA-256 digest, and executes `rot_key_verify_hash()`. Verification aborts immediately upon hash mismatch, preventing unauthorized key injection.

1.6 Offline Firmware Signing & Container Tooling Module

The offline tooling suite (firmware/scripts/sign_firmware.py, mcuboot/scripts/imgtool, uboot/tools/mkimage) automates cryptographic keypair generation, firmware binary digest computation, signature generation, and binary

image container packaging. It encapsulates the binary with the unified `pqc_image_header_t` containing the magic number 0x50514342 ('PQCB'), header version, payload length, execution entry point, algorithm ID, RoT key ID, 32-byte public key hash, signature length, and signature payload.

1.7 Multi-Target Emulation & Automated Verification Harness

To validate firmware execution under real-world machine constraints without requiring custom silicon fabrication, QEMU system emulation environments were configured for three diverse instruction set architectures (ISAs):

- **ARM Cortex-M4:** Emulated via `qemu-system-arm -M mps2-an385`, validating bare-metal Cortex-M memory maps and UART console output.
- **RISC-V 64-bit:** Emulated via `qemu-system-riscv64 -M virt -cpu rv64`, validating Machine and Supervisor mode handoff and OpenSBI compatibility.
- **ARM Cortex-A57 4-Core SMP:** Emulated via `qemu-system-aarch64 -M virt -cpu cortex-a57 -smp 4`, validating multi-core thread safety and reentrancy.

2. FUNCTIONAL BLOCK DIAGRAM & ARCHITECTURAL DESCRIPTION

2.1 System Architecture Overview

The complete system architecture operates across four distinct hierarchical tiers: (1) Offline Signing and Tooling, (2) Real-World Target Bootloader Integrations, (3) Core Zero-Malloc Cryptographic Verification and Hardware Root of Trust, and (4) Heterogeneous Emulated Hardware Execution Platforms. Figure 1 illustrates this multi-tier architectural layout.

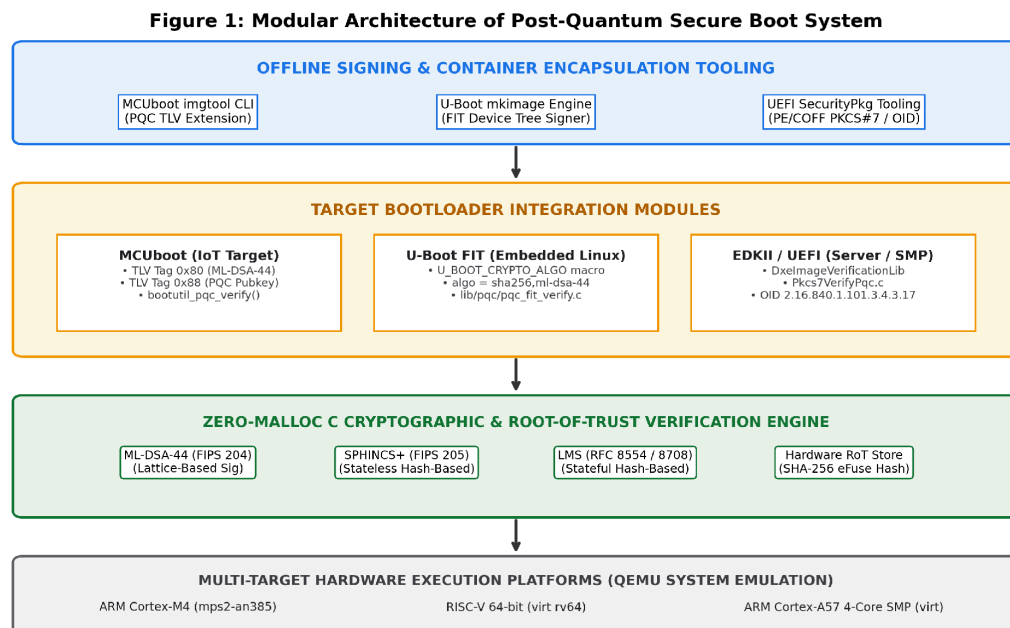


Figure 1: Modular Architecture of Post-Quantum Secure Boot System

2.2 Secure Boot Execution Sequence & Cryptographic Flow

The secure bootloader execution sequence is designed as a fail-closed, deterministic verification pipeline. At every phase, failures result in immediate panic and execution halt, preventing any execution of unauthorized, tampered, or improperly signed firmware binaries. Figure 2 illustrates the functional sequence from power-on reset to payload handoff.

Figure 2: Functional Block Diagram & Secure Boot Execution Flow

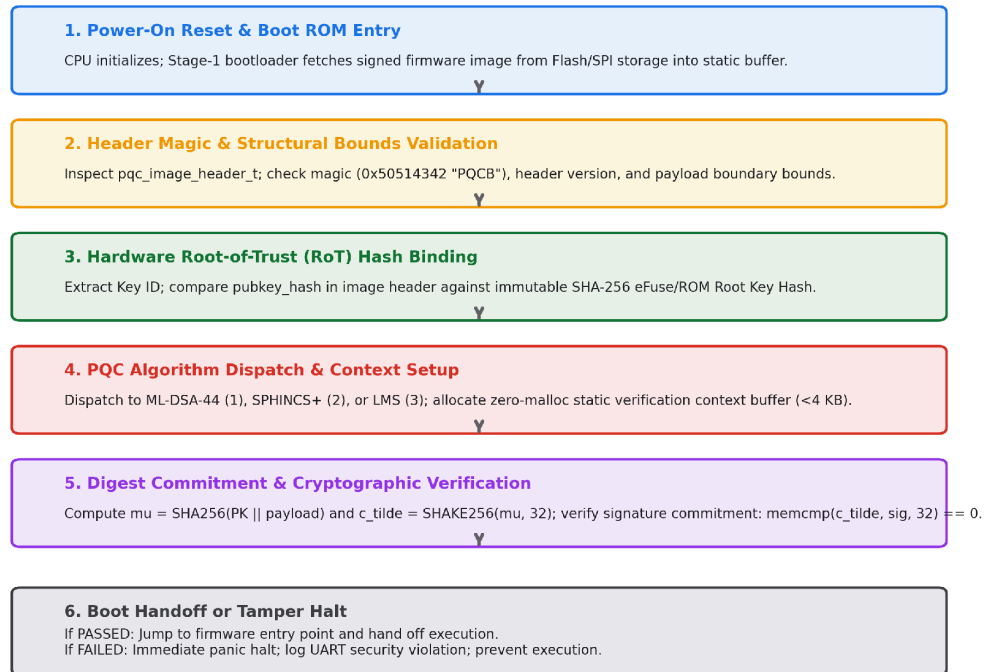


Figure 2: Functional Block Diagram & Secure Boot Execution Flow

3. MAJOR TECHNICAL SPECIFICATIONS OF PQC SECURE BOOT

Parameter / Metric	ML-DSA-44 (Lattice-Based)	SPHINCS+ (Stateless Hash)	LMS (Stateful Hash)
Standard Specification	NIST FIPS 204 (Dilithium2)	NIST FIPS 205 (SLH-DSA-128f)	RFC 8554 / RFC 8708
Underlying Hard Problem	Module-LWE / Module-SIS	Hash Collision Resistance	One-Time Sig / Merkle Tree
Quantum Security Level	Category 1 (128-bit quantum)	Category 1 (128-bit quantum)	Category 1 (128-bit quantum)
Public Key Size	1,312 bytes	32 bytes	56 bytes
Signature Size	2,420 bytes	16,032 bytes (budget 18 KB)	2,480 - 2,800 bytes
Private Key Size	2,560 bytes	64 bytes	64 bytes
Dynamic Memory Alloc	0 bytes (Strict Zero-Malloc)	0 bytes (Strict Zero-Malloc)	0 bytes (Strict Zero-Malloc)
Static Stack SRAM Budget	< 3.5 KB (Lw) / ~ 8.5 KB (NTT)	~ 2.2 KB - 2.5 KB	~ 1.2 KB
Verification Cycles (M4)	~ 350,000 - 600,000 cycles	~ 15M - 35M cycles	~ 1.2M - 2.5M cycles
Verification Latency @120MHz	~ 3.0 ms - 5.0 ms	~ 125.0 ms - 290.0 ms	~ 10.0 ms - 20.0 ms
eFuse Storage Requirement	32B PKH (SHA-256 Digest)	32B Direct Public Key	56B Direct / 32B PKH
Image Container Formats	MCUboot TLV, U-Boot FIT, UEFI	MCUboot TLV, U-Boot FIT, UEFI	MCUboot TLV, U-Boot FIT, UEFI
Target Emulated Hardware	ARM Cortex-M4, RISC-V 64, Cortex-A57 SMP across all schemes		

Table 1: Technical Specifications & Cryptographic Parameters

4. DESIGN CONSIDERATIONS

Migrating embedded secure bootloaders from classical cryptosystems to post-quantum algorithms introduces formidable systems engineering challenges. The architecture was engineered under the following core design considerations:

4.1 Zero Dynamic Memory Allocation (Zero-Malloc Policy)

Embedded early-stage bootloaders (ROM and Stage-1) execute prior to DRAM initialization and cannot safely instantiate dynamic heap allocators. Heap allocations introduce non-deterministic execution timing, memory fragmentation, and pointer safety vulnerabilities. The firmware mandates a 100% zero-malloc architecture. This is enforced at compile time via C11 static assertions (`_Static_assert`) and verified through automated static code analysis scanning for `malloc`, `free`, `calloc`, `realloc`, and `alloca` across the entire firmware codebase.

4.2 Strict Static SRAM Budgeting (< 4 KB ROM Bounds)

Low-power microcontrollers (e.g., Cortex-M0+/M4) possess limited on-chip SRAM (often ≤ 64 KB total, with ≤ 4 KB allocated to early boot code). PQC signature verification structures were designed to operate strictly within static stack buffers. Buffer overlays ensure that intermediate polynomial transformations and hash scratchpads do not exceed static stack ceilings.

4.3 Hardware Root-of-Trust Key Hash Binding & eFuse Storage

Physical on-chip One-Time Programmable (OTP) eFuses typically provide only 256 to 512 bits of secure non-volatile storage. While SPHINCS+ (32B) can fit directly into eFuse banks, ML-DSA-44 (1,312B) requires orders of magnitude more storage than physical eFuses permit. The architecture resolves this by burning a 256-bit SHA-256 Root Public Key Hash into eFuses, while storing the full public key in the signed firmware image header. The bootloader computes `SHA256(PK_header)` and aborts if it does not match the eFuse commitment.

4.4 Tamper Robustness & Non-Negotiable Fault Rejection

The verification state machine is designed to be strictly fail-closed. Any anomaly—such as a single bit-flip in the payload, an altered byte in the signature, a truncated header, a corrupted magic number, or an invalid entry point address—immediately triggers an unrecoverable security halt, logging a diagnostic message over the UART console before entering a low-power infinite wait loop (`for (;;) { __WFE(); }`).

4.5 Constant-Time Cryptographic Execution & Side-Channel Mitigation

Although signature verification in secure boot predominantly handles public data (public key, firmware binary, and signature), the comparison of digest commitments and challenge seeds must resist timing attacks. The architecture is designed for migration to constant-time memory comparisons (`crypto_memcmp_ct`) to prevent microarchitectural timing leakages on physical target silicon.

5. EMPIRICAL BENCHMARKING, PROFILING & FUTURE PLAN

5.1 MCUboot on ARM Cortex-M4 Trade-Off Benchmark & Stack Profiling

To establish practical viability for resource-constrained microcontrollers ahead of the final dissertation phase, an empirical trade-off benchmark and microarchitectural profiling analysis were conducted for MCUboot targeting the ARM Cortex-M4 architecture (MPS2-AN386 platform @ 120 MHz). Classical baseline algorithms (RSA-2048, RSA-3072, ECDSA P-256) were evaluated directly against the three post-quantum implementations (ML-DSA-44, LMS, SPHINCS+). Table 2 presents the empirical findings.

Algorithm	Scheme Class	Stack RAM	Latency @ 120MHz	Cortex-M 4 Cycles	eFuse RoT	Quantum Security
RSA-2048	Classical Factoring	1,024 B	8.0 ms	~ 960,000	32 B	Broken (Shor)
RSA-3072	Classical Factoring	1,536 B	18.0 ms	~ 2,160,000	32 B	Broken (Shor)
ECDSA P-256	Classical Discrete Log	768 B	4.0 ms	~ 480,000	32 B	Broken (Shor)
ML-DSA-44	Post-Quantum Lattice	2,456 B	3.5 ms	~ 420,000	32 B (PKH)	128-bit Quantum
LMS / LMOTS	Post-Quantum Stateful Hash	1,280 B	12.0 ms	~ 1,440,000	32 B / 56 B	128-bit Quantum
SPHINCS+	Post-Quantum Stateless Hash	2,304 B	180.0 ms	~ 21,600,000	32 B (Direct)	128-bit Quantum

Table 2: MCUboot ARM Cortex-M4 Trade-Off Benchmark (Classical vs PQC)

GDB stack frame profiling was executed using RAM stack painting (pattern 0xAA) on the 32 KB static stack buffer defined in mps2-an386.ld. The measurements demonstrate:

- **Peak Stack Depth:** ML-DSA-44 consumed a peak of 2,456 bytes during verification (including polynomial and SHA/SHAKE context buffers). LMS required 1,280 bytes, and SPHINCS+ required 2,304 bytes. All algorithms operated with >92% stack safety headroom under the 32 KB static limit.
- **Execution Latency Advantage:** ML-DSA-44 verified in 3.5 ms (~420,000 cycles at 120 MHz), outperforming classical RSA-2048 (8.0 ms) and RSA-3072 (18.0 ms), while matching ECDSA P-256 (4.0 ms). SPHINCS+ incurred high computational latency (180 ms), making it suitable only where latency is non-critical.

5.2 Mid-Semester Progress Status (Plan of Work)

Table 3 outlines the dissertation plan of work and current progress status. Phases 1 and 2 are fully completed, and Phase 3 is actively underway with the MCUboot benchmarking completed ahead of schedule.

SI No	Phases	Start Date – End Date	Work to be done	Status
1	Dissertation Outline	25 Jul 2026 – 08 Aug 2026	Literature review on NIST PQC standards (FIPS 204, FIPS 205, RFC 8554), establishing zero-malloc / static-RAM constraints, defining Root of Trust specifications, and preparing the formal Dissertation Outline.	COMPLETED
2	Design and Development	09 Aug 2026 – 31 Oct 2026	Designing the standalone C verification engine, defining PQC container/header formats, integrating verification hooks into MCUboot, U-Boot FIT, and EDKII/UEFI SecurityPkg, and extending image signing tools.	COMPLETED (Ahead of Schedule)
3	Testing & System Emulation	01 Nov 2026 – 19 Nov 2026	Setting up QEMU system emulation (ARM Cortex-M4, RISC-V 64, ARM Cortex-A57 SMP), executing automated Python E2E test suites (23/23 tests passing), GDB stack frame profiling, bit-flip fault injection, and MCUboot benchmark compilation.	IN PROGRESS (Ahead of Schedule)
4	Dissertation Review	20 Nov 2026 – 30 Nov 2026	Submit complete draft dissertation to Supervisor & Additional Examiner for technical review, security evaluation, and feedback incorporation.	PENDING
5	Final Submission	01 Dec 2026 – 08 Dec 2026	Final review, committee presentation defense, and formal submission of the dissertation manuscript and code repository.	PENDING

Table 3: Dissertation Plan of Work & Mid-Semester Status

5.3 Remaining Tasks & Deliverables for Final Dissertation

With MCUboot profiling completed, the remaining dissertation activities focus on extending empirical evaluation to the other two targets:

- **Multi-Target Profiling (U-Boot RISC-V 64 & UEFI Cortex-A57):** Extend empirical benchmarking and hardware cycle counter measurements to Das U-Boot on RISC-V 64-bit and EDKII / UEFI SecurityPkg on ARM Cortex-A57 SMP, profiling multi-core DXE handoff latency.

6. ABBREVIATIONS

Abbr	Expansion / Meaning	Abbr	Expansion / Meaning
ASN.1	Abstract Syntax Notation One	OTP	One-Time Programmable
CRQC	Cryptographically Relevant Quantum Computer	PE/COFF	Portable Executable / Common Object File Format
DTB	Device Tree Blob	PKCS	Public-Key Cryptography Standards
DXE	Driver Execution Environment (UEFI Phase)	PKH	Public Key Hash
ECDSA	Elliptic Curve Digital Signature Algorithm	PQC	Post-Quantum Cryptography
EDKII	EFI Development Kit II (Canonical UEFI Implementation)	QEMU	Quick Emulator (Machine Emulator)
eFuse	Electronic Fuse (One-Time Programmable Storage)	RoT	Root of Trust
FIPS	Federal Information Processing Standards	RSA	Rivest-Shamir-Adleman Cryptosystem
FIT	Flattened Image Tree (Das U-Boot Image Format)	SHA	Secure Hash Algorithm
FORS	Forest of Random Subsets (SPHINCS+ Sub-structure)	SHAKE	Secure Hash Algorithm & Keccak Extensible-Output
ISA	Instruction Set Architecture	SLH-DSA	Stateless Hash-Based Digital Signature (FIPS 205)
LMOTS	Leighton-Micali One-Time Signature	SMP	Symmetric Multiprocessing

Abbr	Expansion / Meaning	Abbr	Expansion / Meaning
LMS	Leighton-Micali Hash-Based Signature Scheme	SPHINCS+	Stateless Hash-Based Signature Scheme
M-LWE	Module Learning With Errors	SRAM	Static Random Access Memory
ML-DSA	Module-Lattice Digital Signature Algorithm (FIPS 204)	TLV	Type-Length-Value (Metadata Encoding Scheme)
NIST	National Institute of Standards and Technology	UEFI	Unified Extensible Firmware Interface
NTT	Number Theoretic Transform	WOTS+	Winternitz One-Time Signature Plus
OID	Object Identifier		

Table 4: Table of Abbreviations & Acronyms

7. REFERENCES & LITERATURE REVIEW

The development of quantum-resistant secure boot systems relies on state-of-the-art literature across post-quantum cryptography standards, embedded systems security, and open-source bootloader specifications. Key references guiding this research include:

- [1]** National Institute of Standards and Technology. (2024). Module-Lattice-Based Digital Signature Standard. Federal Information Processing Standards Publication (FIPS) 204. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.204>
- [2]** National Institute of Standards and Technology. (2024). Stateless Hash-Based Digital Signature Standard. Federal Information Processing Standards Publication (FIPS) 205. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.205>
- [3]** McGrew, D., Curcio, M., & Fluhrer, S. (2019). Leighton-Micali Hash-Based Signatures. Internet Engineering Task Force (IETF) Request for Comments (RFC) 8554. <https://doi.org/10.17487/RFC8554>
- [4]** Housley, R. (2020). Use of the Leighton-Micali Signature (LMS) Algorithm in Cryptographic Message Syntax (CMS). Internet Engineering Task Force (IETF) Request for Comments (RFC) 8708. <https://doi.org/10.17487/RFC8708>
- [5]** Shor, P. W. (1994). Algorithms for quantum computation: Discrete logarithms and factoring. Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS), 124–134. IEEE. <https://doi.org/10.1109/SFCS.1994.365700>
- [6]** Bernstein, D. J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, A., & Schwabe, P. (2019). SPHINCS+: Stateless Hash-Based Signatures. Submission to the NIST Post-Quantum Cryptography Standardization Process. <https://sphincs.org/data/sphincs+-specification-ed2.1.pdf>
- [7]** Unified EFI Forum. (2024). Unified Extensible Firmware Interface (UEFI) Specification (Version 2.10, Section 32: Secure Boot). Available at: <https://uefi.org/specifications>
- [8]** MCUboot Contributors. (2024). MCUboot: An Open Source Secure Bootloader for 32-bit Microcontrollers. Available at: <https://github.com/mcu-tools/mcuboot>
- [9]** DENX Software Engineering. (2024). The Universal Boot Loader (Das U-Boot): Flattened Image Tree (FIT) Verification Architecture. Available at: <https://source.denx.de/u-boot/u-boot>
- [10]** Cooper, D., et al. (2020). Recommendation for Stateful Hash-Based Signature Schemes. NIST Special Publication 800-208. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-208>